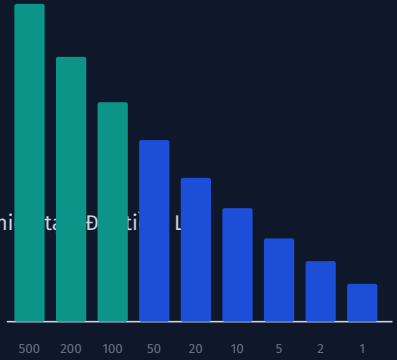
GREEDY C++ NÂNG CAO

Thuật toán Tham lam

Greedy Algorithm

Luôn chọn phương án tốt nhất ở thời điểm hi[illegible] ta[illegible] Đ[illegible]ti[illegible] L[illegible]
lịch và Vắt sữa trong lập trình thi đấu.



Tấn Đức · 01/07/2026

Nội dung bài học

01 · Khái niệm Tham lam

Tối ưu cục bộ dẫn đến tối ưu toàn cục; đặc điểm.

02 · Đổi tiền (Coin Change)

Ý tưởng, code và phản ví dụ $C = \{1, 3, 4\}$.

03 · Lập lịch (Activity Selection)

Chọn cuộc họp kết thúc sớm nhất.

04 · Vắt sữa

Sắp xếp giảm dần, dừng sớm, dùng `long long`.

05 · Kinh nghiệm thi đấu

Dấu hiệu nhận biết, `sort` + `cmp`, phản ví dụ.

06 · Mở rộng & Tổng kết

Cái túi phân số, comparator, độ phức tạp.

Câu chuyện thực tế: Đi siêu thị

Tình huống

Mua đồ hết $15k$, đưa $100k$. Thu ngân cần thối $85k$ sao cho **số tờ ít nhất**.

Bản năng thu ngân

Luôn lấy **mệnh giá lớn nhất có thể** ở thời điểm hiện tại, không nhìn xa hơn.



Mục tiêu bài học

MẢNH GHÉP 01

Hiểu bản chất. Tối ưu cục bộ $\Rightarrow$ tối ưu toàn cục.

MẢNH GHÉP 02

Cài đặt. Sắp xếp + `cmp` cho Đổi tiền, Lập lịch, Vắt sữa.

MẢNH GHÉP 03

Kiểm chứng. Tìm phản ví dụ; nếu sai thì chuyển sang DP hoặc Backtracking.

PHẦN 01

Khái niệm Tham lam

Tối ưu cục bộ, tối ưu toàn cục và các đặc điểm của thuật toán.

Thuật toán Tham lam là gì?

Định nghĩa

Phương pháp giải bài toán tối ưu bằng cách luôn đưa ra **lựa chọn tốt nhất tại thời điểm hiện tại** (tối ưu cục bộ, *local optimum*), với hy vọng dẫn đến kết quả tốt nhất toàn bài (tối ưu toàn cục, *global optimum*).

Đặc điểm

- **Tốc độ:** thường rất nhanh, phụ thuộc bước sắp xếp: $\mathcal{O}(N \log N)$ hoặc $\mathcal{O}(N)$.
- **Bộ nhớ:** $\mathcal{O}(1)$ hoặc $\mathcal{O}(N)$.
- **Thiết kế:** dễ code, tư duy trực quan.

PHẦN 02

Bài toán Đổi tiền

Coin Change: dùng ít tờ tiền nhất với hệ tiền tệ Việt Nam.

Đổi tiền: đề bài và ý tưởng

Đề bài

$C = \{1, 2, 5, 10, 20, 50, 100, 200, 500\}$ (nghìn đồng), số tờ mỗi loại vô hạn. Đổi số tiền S bằng **ít tờ nhất**. **Input:** $S = 285$ **Output:** 5 (200, 50, 20, 10, 5)



Kết quả

$285 = 200 + 50 + 20 + 10 + 5 \Rightarrow$ **5 tờ.**

Cài đặt bài toán Đổi tiền

```cpp
#include <iostream>
#include <vector>
using namespace std;

int main() {
    // Mệnh giá giảm dần
    vector<int> coins = {500, 200, 100,
                         50, 20, 10, 5, 2, 1};
    int S = 285, ans = 0;
    for (int c : coins) {
        ans += S / c; // lấy tối đa tờ c
        S %= c;       // tiền còn lại
    }
    cout << ans << endl;
}
```

Đọc code

- `S / c`: số tờ tối đa mệnh giá `c`.
- `S %= c`: phần tiền còn phải đổi.

Kết quả

Với $S = 285$ in ra **5**.

Khi nào Tham lam thất bại?

Phản ví dụ: $C = \{1, 3, 4\}$, $S = 6$

Tham lam $6 = 4 + 1 + 1$ (3 tờ)

Tối ưu thực sự

Tối ưu $6 = 3 + 3$ (2 tờ)

Quy tắc

Tham lam **không** đúng cho mọi hệ tiền tệ; chỉ đúng với hệ "chuẩn" (*canonical coin systems*). Với hệ bất kỳ, dùng **Quy hoạch động** (Dynamic Programming).

PHẦN 03

Bài toán Lập lịch

Activity Selection: tối đa số cuộc họp trong một phòng họp.

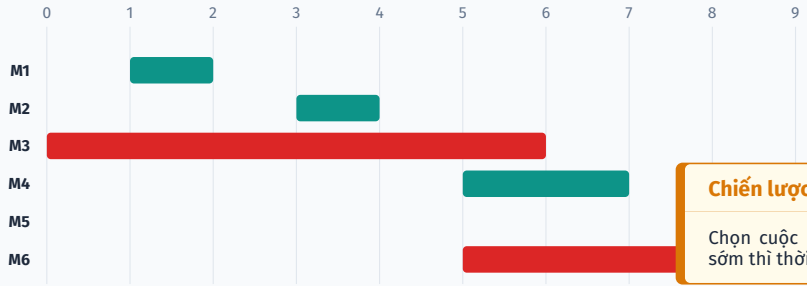
Lập lịch: đề bài

Đề bài

N cuộc họp, cuộc họp i bắt đầu tại S_i, kết thúc tại E_i. Một phòng họp chỉ tổ chức một cuộc họp tại một thời điểm. Tìm **số cuộc họp tối đa**.

Cuộc họp	1	2	3	4	5	6
Bắt đầu (S_i)	1	3	0	5	8	5
Kết thúc (E_i)	2	4	6	7	9	9

Lập lịch: chiến lược chọn kết thúc sớm nhất



Chiến lược tối ưu

Chọn cuộc họp **kết thúc sớm nhất**: càng sớm thì thời gian trống còn lại càng nhiều.

Kết quả

chọn M1, M2, M4, M5 ⇒ 4 cuộc họp. **bỏ** M3, M6 (chồng lấn).

Cài đặt bài toán Lập lịch

Ý chính

- Sắp xếp theo E_i tăng dần.
- Hợp lệ khi `start` $\geqslant$ `last_end`.

```cpp
#include <bits/stdc++.h>
using namespace std;
struct Meeting { int start, end; };
bool cmp(const Meeting &a, const Meeting &b) {
    return a.end < b.end;
}
int main() {
    vector<Meeting> a = {{1,2},{3,4},{0,6},
                         {5,7},{8,9},{5,9}};
    sort(a.begin(), a.end(), cmp);
    int cnt = 0, last_end = -1;
    for (auto m : a)
        if (m.start >= last_end) {
            cnt++; last_end = m.end;
        }
    cout << cnt << endl;
}
```

PHẦN 04

Bài toán Vắt sữa

Sắp xếp giảm dần để lượng sữa hao hụt ít nhất.

Vắt sữa: đề bài

Tóm tắt đề bài

Đàn bò n con, con i dự kiến vắt được a_i lít. Mỗi lần vắt một con, các con còn lại **chưa được vắt** bị giảm đúng 1 lít (sản lượng ≤ 0 thì vắt được 0). Xác định thứ tự vắt để tổng sữa **nhiều nhất**.

Input	Dãy a_i	Output
4	4 4 4 4	10
4	2 1 4 3	6

Vắt sữa: ý tưởng Tham lam

Nhận xét

Vắt con bò ở lượt k (từ lượt 0) thu được $\max(0,\, a_i - k)$ lít.

Ý tưởng

Ưu tiên con có **sản lượng lớn nhất** trước.

1. Sắp xếp A giảm dần.
2. Duyệt $i = 0, \ldots, n-1$, cộng $\max(0, A[i] - i)$.
3. Nếu $A[i] - i \le 0$: dừng sớm.

Ví dụ 2 1 4 3, sau sắp xếp: 4 3 2 1

i	$A[i]$	$A[i] - i$	Cộng
0	4	4	4
1	3	2	2
2	2	0	dừng

Tổng = 6

Code mẫu: Vắt sữa

```cpp
void solve(vector<long long> &a, int n) {
    sort(a.begin(), a.end(),
         greater<long long>());
    long long max_milk = 0;
    for (int i = 0; i < n; ++i) {
        if (a[i] - i <= 0) break; // dừng sớm
        max_milk += a[i] - i;
    }
    cout << max_milk << "\n";
}
```

Edge case

Tổng tối đa $n \times \max(a_i) = 10^5 \times 10^9 = 10^{14}$, vượt `int` 32-bit: **bắt buộc** dùng `long long` cho biến tổng.

Sửa nhỏ so với slide gốc

Dùng `vector<long long>` khớp với `greater<long long>()`.

PHẦN 05

Kinh nghiệm thi đấu

Competitive Programming: nhận biết, cài đặt và kiểm chứng.

Mẹo thi đấu CP

Dấu hiệu nhận biết

Bài yêu cầu tìm lớn nhất, nhỏ nhất, ít nhất, nhiều nhất; các lựa chọn độc lập và hướng tới một hàm mục tiêu đơn giản.

Chìa khóa: Sắp xếp (Sort)

Hầu như bài Greedy nào cũng cần hàm `cmp()` tùy biến để `sort` theo một tiêu chí (kết thúc sớm nhất, tỷ lệ lớn nhất, ...).

Kiểm chứng bằng phản ví dụ

Dành 10 giây tìm test nhỏ phá vỡ ý tưởng trước khi code. Nếu có, chuyển sang **DP** hoặc **Quay lui (Backtracking)**.

PHẦN 06

Mở rộng

Cái túi phân số, comparator và bảng độ phức tạp.

Mở rộng Cái túi phân số (Fractional Knapsack)

```cpp
struct Item { double w, v; };
bool cmp(const Item &a, const Item &b) {
    return a.v / a.w > b.v / b.w;
}
double knapsack(vector<Item> &it, double W) {
    sort(it.begin(), it.end(), cmp);
    double res = 0;
    for (auto &x : it) {
        if (W <= 0) break;
        double t = min(x.w, W);
        res += t * x.v / x.w;
        W -= t;
    }
    return res;
}
```

Ý tưởng

Sắp xếp theo **tỷ lệ giá trị/khối lượng** giảm dần, lấy trọn từng vật, vật cuối lấy một phần cho đầy túi.

Lưu ý

Vật **không** chia nhỏ được (0/1) thì Tham lam không còn đúng; dùng DP.

Comparator và độ phức tạp

Sai: $\leqslant$ trong cmp

Vi phạm *Strict Weak Ordering*: `sort` có thể crash hoặc hành vi không xác định (UB).

Đúng: dùng < hoặc >

So sánh chặt: `a.end < b.end`. Có thể viết bằng lambda; cần giữ thứ tự bằng nhau thì dùng `stable_sort`.

Bài	Bước chính	Thời gian
Đổi tiền	Duyệt mệnh giá giảm dần	$\mathcal{O}(k)$, k = số mệnh giá
Lập lịch	Sắp xếp theo E_i	$\mathcal{O}(N \log N)$
Vắt sữa	Sắp xếp giảm dần	$\mathcal{O}(N \log N)$

Tổng kết bài học

3 trụ cột

1. **Bản chất:** tối ưu cục bộ dẫn đến tối ưu toàn cục.
2. **Công cụ:** `sort` + `cmp`, $\mathcal{O}(N \log N)$.
3. **Kiểm chứng:** tìm phản ví dụ, sai thì chuyển DP.

Lỗi kinh điển

× Dùng Tham lam cho hệ tiền không chuẩn ($C = \{1, 3, 4\}$).
× Quên `long long` khi tổng đạt 10^{14}.
× Code ngay mà không thử phản ví dụ.

Cảm ơn các bạn đã theo dõi! Chúc các bạn code đúng và nhanh.